

What is security testing?

We can distinguish vulnerability assessments, penetration testing and red teaming.

1. The first one is targeted to systems/networks, it's interested in finding as many vulnerabilities as possible, without going super deep. It's primarily an automated approach. The main limitation is that it does not find a lot of attacks paths.
2. *Got lost*. It's not a one-shot approach, and needs to be done periodically.
3. It deals with emulation of real world attackers. It does not only try to find vulnerabilities to get into systems and prove that certain attacks are possible, but they also emulate threat actors, like APTs (Advanced Persistent Threat). Typically, there are some specific hackers for specific companies.

When performing these operations, ethical hackers needs to be careful.

## **Kill chain & ATT&CK**

It's a framework, showing attacker journeys, from initial reconnaissance to getting sensitive data or doing harm. Understanding each stage allows us to implement a defense for each stage. The most popular kill chain is given by Lockheed Martin KC, made by 7 steps.

1. Reconnaissance
2. Weaponization, we prepare our tools to exploit certain targets.
3. Delivery
4. Exploitation
5. Installation
6. Command and Control (C2)
7. Actions and Objective

Some of these steps could be skipped, they are not always mandatory.

*Read, from slides, explanations for each step.*

Command and Control is the stage that you get when you compromise a system, and want to maintain a persistent channel with the system. It's important as the C2 facility (which is in some form a reverse shell) needs to take into account operational security. *Take a look at this.*

The ATT&CK is also a framework, published by MITRE, providing a taxonomy of adversaries actions. It maps techniques to specific APT groups, so techniques used by specific attackers.

As a red team, if we want to emulate APTs for a specific company we need to know what they do, so it's quite useful.

## How to practice

Occasionally in labs there will be some activities. We should also practice at home with Hack The Box, and so on. We should create a virtualized environment to test vulnerabilities. We can pick VirtualBox, but also KVM (for Linux).

Try Try Hack Me for easier boxes to do. Install a virtual machine (like Kali Linux, Parrot Os) with VPN capabilities and docker. A network interface in our machine should let us connect to the internet, specifically to Hack the Box or Try Hack Me.

To get started, we can use Metasploitable 2. It's a machine that we can use and try to compromise it. Metasploitable machine can be used for an internal network, to avoid having to deal with internet at first.

The docker environment is going to be inside the attack box.

Also, we can use Juice-shop as vulnerable machine.

## Vulnerabilities

Let's talk about common vulnerabilities and exposures, CVE. It's useful both for Red team and Blue team. CVE is a standardised reference of known vulnerabilities. Each CVE is assigned with a score, the CVSS. It stands for the severity measures. Most of CVSS is v3.1. The score is given by 8 attributes for the base score, 3 attributes for temporal score (how much the vulnerability has been around), 11 attributes for environmental score.

The CVE score does not tell us the risk, as:

If the likelihood is zero, the risk is zero, even if the CVE has a score of .

## Common Weakness Enumeration

How do we categorise vulnerabilities? We can do it with CWE (Common Weakness Enumeration), a common language. There are some syntactical rules on how to write the "report" for the vulnerability.

*Check what KEV is.*

The score for CWEs is a risk measure, not impact/severity. For example, the top

one "Use after free" has a score of 73.99. A lot of risky vulnerabilities are related to memory corruption, like heap-based buffer overflow.

## Searching Vulnerabilities

How to search for known vulnerabilities for a particular software? There are databases containing vulnerabilities, like CVEs databases (NVD). The CVEs lists vulnerable software, so a list of well-formed names, including versions of software, name of the software, and other information.

When searching for vulnerabilities, we need to provide info a search engine can understand. For something automated, we need to use those well-formed names, though just the name and version should be fine when doing this manually. Github could also be a trustworthy source. Also, we can run `nmap --script vulnrs` if using Metasploitable, for example. It gives back the list of potential vulnerabilities a software is affected by.

*Missing part*

## Enumeration

A powerful tool is given by `nmap`, it can uncover details beyond basic network presence, like versions, protocols, configurations. We can also use `--reason` flag, telling us why a certain information is given to us by `nmap`.

The `-sV` tag can show additional information, but in case of an SSH server it could be trivial, as we immediately get the banner back.

NMap also has the Nmap Script Engine (NSE), and categorizes the script it can run in several categories:

- safe: connecting and doing passive analysis.
- intrusive: could potentially crash the target, like sending high volumes of packets.
- exploit: perform active exploitation. We can write our own script in `lua`.
- brute: trying default credentials
- discovery: attempt to query

There are 604 predefined NSE scripts. Some of these scripts takes arguments, like having valid credentials for some vulnerabilities.

## Practical Demo

Let's start with a Kali Linux machine.

First thing, install `metasploit`, and then start a `msfconsole`.

What is metasploit? It's a framework offering exploits, modules, encoders, and evasion techniques.

Let's start with a database creation process. In real world, we want to keep track of what we're doing. Database will collect all information that we see on screen (going through several stages).

So, let's start the database with `sudo msfdb reinit` (or something like this) (outside msfconsole I think).

Another important concept is workspaces, like having a workspace for each company, or having a room for HackTheBox, and one for TryHackMe. Launch `workspace` to showing active rooms.

Then with `db_nmap` we launch a `nmap` command, listing potential vulnerabilities with a severity score.

Now, I can query the database and check some things:

- `hosts` tell us the addresses
- `services` shows services that were running
- `vulns` the vulnerabilities (possible ones).

After doing a search on the database, we can use the `search` command, select the entry (among the exploits), and use `use n_entry`.

Also, we can use `options` after selecting an exploit, we have all the options that we have to specify to use that exploit. Then we set the values with `set`, and then `run`.